

语法分析器实验报告

姓名：程宣赫

学号：2023202250

2026 年 4 月 23 日

1 实验思路

1.1 整体任务框架分析

语法分析是编译过程中的核心环节之一，其任务是在词法分析的基础上，根据给定文法判断输入串是否构成合法的程序结构。词法分析所解决的是“字符序列如何组成单词符号”的问题，而语法分析进一步研究的是“单词符号如何按照文法规则组成语句、声明、表达式以及完整程序”的问题。其核心聚焦的是文法，以及文法的检查是如何在代码里实现的。

因此，本实验的核心目标也就是，在对一段程序进行词法分析结果的基础上，对词法分析器传入的 token 流进行语法层面的分析，并判断其是否符合给定的 SysY 语言文法。若输入串符合文法，则语法分析器需要进一步给出相应的分析结果，即归约过程中使用的文法规则序列以及对应的语法树；若输入串不符合文法，则语法分析器需要尽可能定位错误，并在一定条件下继续分析后续输入。

本实验面向的是 SysY 语言子集的文法描述。在实验指南给出的 SysY 语言定义中，采用 EBNF 形式给出了 SysY 语言的文法规则，用一系列的终结符、非终结符、开始符号以及产生式规则，描述了程序中声明、语句、表达式以及函数定义等结构的组成方式。

此外，本实验采用 Yacc 作为语法分析器生成工具，其理论基础进一步具体化为自底向上的 LR 分析方法。与自顶向下分析从开始符号出发尝试推导输入串不同，自底向上分析从输入符号串出发，通过不断识别句柄并执行归约，逐步将输入串归约为开始符号。换言之，分析过程本质上是一个“移进—归约”过程：在适当时机将当前输入符号移入分析栈，并在栈顶符号串匹配某条产生式右部时，将其归约为对应的左部非终结符。

若最终整个输入能够被归约为开始符号，则说明输入在语法上是合法的；否则，说明输入不符合给定文法。

基于上述理论背景，可以进一步明确，本实验在实现层面需要解决的几个核心问题。首先，实验指南中给出的 SysY 文法采用 EBNF 形式描述，其中包含可选项、重复项等扩展写法，而 Yacc 只能直接处理以产生式规则表示的上下文无关文法，因此必须先对原始文法进行改写，将其转换为适合 Yacc 处理的 BNF 格式的文法形式。其次，语法分析器并不直接面对源程序字符流，而是建立在词法分析器输出的 token 序列之上，因此还需要解决词法分析器与语法分析器之间的接口对齐问题，即如何使词法分析结果能够以语法规则中一致的终结符形式传递给 Yacc，并在必要时保留标识符、常量等符号的附加信息。更进一步地，本实验并不仅要求完成语法正确性的判断，还要求输出归约过程中使用的语法规则序列，并构建对应的语法树，因此需要进一步考虑如何在分析过程中同步维护结构化结果，使归约动作与树形表示能够保持一致。此外，程序设计语言中的表达式通常包含多层优先级与结合性，而条件语句中又存在典型的悬挂 `else` 二义性，因此还必须通过合理的文法组织与优先级处理保证分析结果符合语言定义。最后，考虑到实验输入中可能包含语法错误，语法分析器还应具备一定的错误诊断与局部恢复能力，从而避免在首个错误处立即终止，并尽可能继续分析后续输入。

1.2 整体执行流程与功能划分

在明确了语法分析实验所涉及的理论问题之后，整个系统的实现过程实际上可以进一步分解为若干前后衔接的处理阶段，即由文法准备、接口建立、分析执行、结构输出以及错误处理共同组成的完整过程。

1.2.1 文法整理与改写

整个流程的起点并不是直接编写 Yacc 规则，而是先对实验指南中给出的 SysY 文法进行整理。实验材料采用 EBNF 形式描述语言规则，其中包含可选项、重复项等扩展写法。这种写法在理论描述中较为紧凑，但 Yacc 只能直接处理以普通产生式规则表示的上下文无关文法，因此必须先将原始文法改写为等价的 BNF 风格文法。

在本实验中，最典型的改写首先体现在“重复结构”的展开上。例如，实验指南中的常量声明规则可写为：

$$\text{ConstDecl} \rightarrow \text{const BType ConstDef} \{, \text{ConstDef}\};$$

其中花括号表示“重复零次或多次”，这一写法不能直接用于 Yacc。因此，在实现中需要将其改写为递归形式，即先保留一个基本的 `ConstDef`，再引入列表结构表示后续重复出现的部分。改写后可表示为：

$$\text{ConstDecl} \rightarrow \text{CONST BType ConstDefs SEMICOLON}$$

$$\text{ConstDefs} \rightarrow \text{ConstDef}$$

$$\text{ConstDefs} \rightarrow \text{ConstDefs COMMA ConstDef}$$

这样便将原先 EBNF 中的重复部分转化为了 Yacc 可以直接处理的左递归列表结构。与此类似，变量定义列表、初始化值列表、函数形参与实参列表等规则，也都采用了相同的改写思路。

另一类典型改写是“可选结构”的展开。例如，实验指南中的编译单元规则可写为：

$$\text{CompUnit} \rightarrow [\text{CompUnit}] (\text{Decl} \mid \text{FuncDef})$$

其中方括号表示“可有可无”。这一形式在 Yacc 中同样不能直接保留，因此需要引入新的非终结符，将“一个或多个编译单元成分”的结构显式写出。实验中将其改写为：

$$\text{CompUnit} \rightarrow \text{CompUnitItems}$$

$$\text{CompUnitItems} \rightarrow \text{CompUnitItem}$$

$$\text{CompUnitItems} \rightarrow \text{CompUnitItems CompUnitItem}$$

$$\text{CompUnitItem} \rightarrow \text{Decl}$$

$$\text{CompUnitItem} \rightarrow \text{FuncDef}$$

经过这一改写后，原本 EBNF 中通过可选项和组合形式表达的结构，被拆解成了更加标准的上下文无关语法规则。

除了重复与可选结构之外，数组维度相关规则也需要改写。例如，实验指南中的常量定义规则可写为：

$$\text{ConstDef} \rightarrow \text{Ident} \{ [\text{ConstExp}] \} = \text{ConstInitVal}$$

其中数组维度部分是一个可重复结构，表示可以有零个、一个或多个下标。实验中将其改写为：

$$\text{ConstDef} \rightarrow \text{IDENT ConstArrayDims ASSIGN ConstInitVal}$$

$$\text{ConstArrayDims} \rightarrow \varepsilon$$

$$\text{ConstArrayDims} \rightarrow \text{ConstArrayDims ConstArrayDim}$$

$$\text{ConstArrayDim} \rightarrow \text{LBRACK ConstExp RBRACK}$$

这样处理之后，数组维度信息就不再依赖 EBNF 中的重复记号，而是转化为了可由分析器逐步归约的文法层次。

函数参数中的数组形式同样需要类似处理。实验指南中对应规则为：

$$\text{FuncFParam} \rightarrow \text{BType Ident} \left[\left[\right] \left\{ \left[\text{Exp} \right] \right\} \right]$$

在实验实现中，这一规则被进一步拆分为函数参数本体与数组后缀两部分，从而转化为 Yacc 可处理的形式。这样的改写虽然使规则数量有所增加，但能够更清晰地刻画参数是否为数组、是否具有后续维度等不同情况。

在文法改写之外，对于语法二义性问题，其中最典型的是悬挂 `else` 问题。对于形如 `if (a) if (b) S_1 else S_2` 的输入，`else` 的归属若不加以限定，就会产生二义性。

本实验借助 Yacc 中的优先级声明机制对该问题进行处理。具体而言，在 Yacc 中首先定义两个优先级：`LOWER_THAN_ELSE` 和 `ELSE`，并使后者具有更高优先级。在语法规则中，对于不带 `else` 的 `if` 语句，通过 `%prec LOWER_THAN_ELSE` 显式赋予其较低优先级，而带 `else` 的规则则自然具有更高优先级。

与悬挂 `else` 不同，表达式中的优先级问题在本实验中并不主要依赖额外补充规则，而是依赖实验指南原始文法本身已经给出的层次划分。例如，原始文法已经将乘除模运算组织为 `MulExp`，将加减运算组织为 `AddExp`，再进一步向上构成关系表达式、相等表达式、逻辑与表达式和逻辑或表达式。这样一来，对于 `a + b * c`，文法会自然地先在 `MulExp` 层处理 `b * c`，再在 `AddExp` 层处理整体加法；对于 `a <= 10 || b > 2 && c < d`，文法也已经通过 `LAndExp` 与 `LOrExp` 的层次区分体现了逻辑与优先于逻辑或的关系。因此，在实验实现中，这部分工作的关键并不是重新设计优先级，而是在改写文法时保持原始层次结构不被破坏。

1.2.2 从词法分析器到语法分析器

在完成文法整理之后, 下一步需要解决的是词法分析器与语法分析器之间的接口问题。语法分析器并不直接处理源程序字符流, 而是建立在词法分析器输出结果的基础上。因此, 词法分析器识别出的各类单词符号, 在进入语法分析阶段后, 都应当对应为文法中的终结符。只有当词法分析结果与语法规则中使用的终结符保持一致时, 语法分析器才能根据文法规则对输入进行归约。

在原有词法分析实验中, 词法分析器的主要任务是识别每个单词的类别, 并输出其属性和值所在位置等信息, 其输出形式更偏向于词法分析结果展示。而在本实验中, 必须对原有的 `lexer.l` 进行改写。

具体而言, 改写后的词法分析器在识别关键字、运算符和界符时, 不再输出词法类别标记, 而是直接返回语法分析器中声明的终结符。例如, 识别到 `const`、`int`、`if` 等关键字时, 应分别返回对应的关键字终结符; 识别到加减乘除、关系运算符、括号、分号和逗号等符号时, 也应返回与文法规则中一致的运算符终结符和界符终结符。这样一来, 词法分析器输出的结果就不再是单纯的文本信息, 而是能够被语法分析器直接接收和处理的符号流。

对于标识符和整型常量这类符号, 仅返回终结符类型还不够, 因为后续语法树构建还需要保留它们的具体文本内容。因此, 在改写过程中, 还需要为这类终结符附加语义值传递机制, 使词法分析器在返回 `IDENT` 和 `INT_CONST` 的同时, 将对应的词素内容继续传递给语法分析器。这样, 语法分析器在执行归约动作时, 就不仅能够知道当前符号属于哪一类, 还能够获取其实际名称或字面值, 从而为终结符节点构造提供基础。

在语法分析器一侧, 这些终结符首先需要在 `parser.y` 中统一声明。也就是说, 词法分析器返回的每一种记号, 都必须在语法分析器中以终结符的形式显式给出。例如, `CONST`、`INT`、`VOID`、`IF`、`ELSE` 等关键字需要被声明为终结符, `PLUS`、`MINUS`、`MUL`、`DIV`、`ASSIGN`、`EQ` 等运算符也需要被声明为终结符。对于 `IDENT` 和 `INT_CONST` 这类除类别之外还需要保留具体文本内容的符号, 则还需要在终结符声明的同时, 为其指定对应的语义值类型。

在终结符声明完成之后, 语法分析器还需要进一步声明 EBNF 文法中出现的非终结符, 以及为了将 EBNF 转换为 BNF 而新引入的辅助非终结符。终结符表示的是词法分析阶段能够直接识别出来的基本符号, 而非终结符则表示语法分析阶段所关心的结构单位, 例如编译单元、声明、语句、表达式、函数定义等。在 `parser.y` 中, 这些非终结符通过 `%type` 逐一给出, 并与相应的语义值类型建立对应关系, 从而使它们在归约完成后能够承载进一步的结构信息。在本实验中, 由于需要在归约过程中同步构造语法

树，因此所有非终结符都对应统一的树节点类型，这样在每一次归约发生时，当前规则左部都能够形成一个新的结构节点。

随后 `parser.y` 需要对每一条 BNF 格式下的产生式进行具体定义。对于 Yacc 而言，一条完整的文法规则不仅包含产生式左部和右部，还通常需要在规则后附加相应的语义动作。前者用于描述语法结构本身，即哪些终结符和非终结符可以归约为更高层次的非终结符；后者则用于规定当该条规则被成功归约时，语法分析器应执行何种处理。例如，在本实验中，语义动作不仅需要记录当前归约所对应的文法规则，还需要根据规则右部成分构造相应的语法树节点，并维护各节点之间的父子关系。也就是说，`parser.y` 中的每条产生式实际上都同时承担了“描述文法结构”和“定义归约后处理逻辑”这两项功能。

因此，从语法分析器的角度看，整个规则系统实际上是由三部分共同构成的：第一部分是终结符声明，它规定了词法分析器能够向语法分析器提供哪些基本符号；第二部分是而非终结符声明，它规定了语法分析阶段需要刻画哪些结构层次；第三部分是产生式规则本身，它规定了这些终结符和非终结符之间如何组合与归约。只有这三部分在 `parser.y` 中被统一组织起来，语法分析器才能真正根据输入 token 流完成后续的移进、归约以及结构构造过程。

1.2.3 归约结果维护与语法树结构设计

更进一步地，本实验并不仅要求完成语法正确性的判断，还要求输出归约过程中使用的文法规则序列，并构建对应的语法树。因此，语法分析器在执行每一次归约时，不能只完成“是否接受某条规则”的判断，还必须同步维护分析结果的结构化表示，使归约过程与语法树构造保持一致。这也是本实验相比单纯语法判定更进一步的地方。

为了实现这一目标，实验中首先引入了统一的树节点结构 `Node`。从功能上看，这一结构作为整个语法树的基本组成单元，用于统一表示终结符节点和非终结符节点。每个节点至少需要记录三个方面的信息：其一是节点名称，用于表明该节点对应的是哪一个终结符或非终结符；其二是附加文本信息，用于在必要时保存标识符名称、常量字面值或特定终结符的实际内容；其三是子节点集合，用于刻画当前节点与下层结构之间的父子关系。

在此基础上，为了能够对语法树进行动态维护，实验中进一步围绕 `Node` 设计了一组辅助操作。首先，需要有专门的节点创建操作 `new_node()`，用于在归约过程中根据当前规则左部或当前识别到的终结符生成新的树节点。其次，需要有子节点挂接操作 `add_child()`，用于将已经构造好的下层节点追加到当前节点之下，从而逐步形成完整

的父子层次关系。再次，还需要有树遍历与导出相关的操作，使已经构造完成的树能够输出得到 .dot 文件。

综上所述，在本实验中，Node 及其相关操作构成了归约结果维护与语法树构造的核心基础。通过引入统一的节点结构，并在每条产生式的语义动作中同步完成规则记录和节点组织，语法分析器得以将线性的 token 序列逐步转换为具有层次关系的树形结构，同时保留分析过程中实际使用到的文法规则序列，从而使实验结果兼具过程可追踪性和结构可视化特征。

1.2.4 错误处理机制设计

在实际语法分析过程中，输入程序往往不一定完全符合语法规则。如果语法分析器在遇到第一个错误时立即终止分析，那么不仅无法提供有效的错误定位信息，也无法继续分析后续结构，从而降低了分析器的实用性。因此，在本实验中，除了完成语法正确性的判断外，还设计了一定程度上较为简单的错误处理机制，使分析器在出现局部错误时能够尽可能恢复并继续分析。

从 Yacc 的机制来看，错误处理主要依赖其内置的 error 符号。当输入符号流无法匹配当前语法规则时，分析器会进入错误状态，此时可以通过在文法中显式引入包含 error 的产生式，使分析器弹出栈内的内容，找到第一个能匹配带 error 的文法规则的栈内容后，跳过出现 error 后的部分非法输入，并在合适的位置恢复正常分析过程。

在本实验中，简单错误处理主要集中在几类典型语法结构上，例如声明语句和普通语句。这类结构通常以分号作为结束标志，因此可以利用这一特征，将错误恢复点设置在分号处。以变量声明为例，在正常规则之外，可以补充形如

$$\text{VarDecl} \rightarrow \text{BType error SEMICOLON}$$

的错误恢复规则；对于常量声明，也可以补充形如

$$\text{ConstDecl} \rightarrow \text{CONST BType error SEMICOLON}$$

的规则。这样，当声明结构在类型之后出现无法继续匹配的部分时，分析器可以跳过当前错误片段，并在遇到分号后恢复分析。

在 Stmt 语句层面，错误处理同样采用类似思路。由于普通语句也常以分号结束，因此可以加入形如

$$\text{Stmt} \rightarrow \text{error SEMICOLON}$$

的恢复规则。该规则表示当当前语句无法按照正常语句规则继续归约时，分析器可以将错误限制在当前语句范围内，并以分号作为同步点继续后续分析。通过这些错误恢复规则，分析器能够在部分局部错误出现时继续工作，而不是在第一次语法错误处直接终止。

由于引入错误处理不仅可能会使文法变得冗余复杂，还可能引入新的冲突。在本实验中，仅针对“容易出现错误且具有明确恢复边界”的结构（如以分号结束的语句）设计错误恢复规则，从而在保证分析器稳定性的同时，控制文法复杂度。

2 实验过程中遇到的问题与解决方法

2.1 int 归约导致的 R-R 冲突问题

2.1.1 问题介绍

在按照实验指南给出的 SysY 文法进行改写后，语法分析器在生成过程中出现了 reduce/reduce conflict，即 R-R 冲突。该问题的核心原因在于，变量声明和函数定义都可能以 `int` 开头，并且在最初的文法设计中，`int` 同时可以被归约为 `BType` 和 `FuncType`。

具体而言，变量声明通常具有如下形式：

$$\text{VarDecl} \rightarrow \text{BType VarDefs SEMICOLON}$$

而函数定义则具有如下形式：

$$\text{FuncDef} \rightarrow \text{FuncType IDENT LPAREN} \dots$$

其中，`BType` 可以推出 `INT`，`FuncType` 也可以推出 `INT`。因此，当分析器读到 `INT` 时，可能同时面临两种归约选择：将其归约为 `BType`，或者将其归约为 `FuncType`。这就导致了归约之间的冲突。

该问题在处理形如 `int main()` 的函数定义时表现得较为明显。分析器在看到 `int` 后如果过早将其按照变量声明方向处理，则在后续遇到左括号时可能无法继续匹配变量声明规则，从而导致语法分析失败。因此，该问题并不是单纯的警告，而是会直接影响 `int` 返回值函数定义的识别。

2.1.2 解决办法

解决这一问题的关键在于避免让 `INT` 同时归约为两个不同的类型非终结符。实验中采用的处理方式是取消单独的 `FuncType` 对 `INT` 的归约，将函数定义规则改写为直接使用 `BType` 表示返回类型为 `int` 的函数，同时保留 `VOID` 作为另一类函数返回类型。

改写后的函数定义规则可以表示为：

$$\text{FuncDef} \rightarrow \text{BType IDENT LPAREN FuncFParams RPAREN Block}$$

$$\text{FuncDef} \rightarrow \text{VOID IDENT LPAREN FuncFParams RPAREN Block}$$

这样，`INT` 只会先归约为 `BType`，不再同时存在 `FuncType` 与 `BType` 两条竞争性的归约路径。对于 `int` 开头的结构，分析器随后可以根据 `IDENT` 后面的符号继续区分变量声明和函数定义：若后面是左括号，则按照函数定义处理；若后面是赋值、数组下标、逗号或分号等，则按照变量声明处理。

通过这种改写，冲突的根源被消除，文法结构也更加统一。与此同时，函数定义中的返回类型仍然可以在语法树中正常体现，因为 `BType` 本身会进一步包含 `INT` 终结符节点。因此，这一修改既解决了 R-R 冲突，又没有破坏语法树对函数返回类型的表示。

2.2 空产生式与语法树叶子节点问题

2.2.1 问题介绍

在构建语法树时，实验最初采用的做法是：无论产生式是否为空，都为其左部非终结符建立一个对应节点。这样处理虽然能够完整保留文法层次，但会带来一个问题：当某个空产生式被使用时，其对应的非终结符节点没有任何子节点，从而成为语法树中的叶子节点。

例如，对于数组维度列表，可以有如下规则：

$$\text{ConstArrayDims} \rightarrow \varepsilon$$

如果在该规则对应的语义动作中直接建立 `ConstArrayDims` 节点，那么在变量或常量没有数组维度时，语法树中就会出现一个没有子节点的 `ConstArrayDims`。这类节点本身是非终结符，但却出现在叶子位置，与实验中“叶子节点应当对应终结符”的要求不一致。

类似的问题也会出现在函数参数列表、实参列表、语句块内容列表、下标列表等可空结构中。由于这些规则本身用于表示“可以没有某一部分”，如果直接将空产生式也构造成树节点，就会在语法树中产生大量无实际输入符号对应的非终结符叶子，影响语法树结构的清晰性。

2.2.2 解决办法

为了解决这一问题，实验中对空产生式采取了特殊处理：空产生式不再建立独立的语法树节点，而是返回空指针。由于子节点挂接函数在添加子节点时会判断待添加节点是否为空，因此空产生式不会被真正挂入语法树中。这样可以避免空非终结符成为叶子节点，使最终语法树中的叶子节点主要对应实际输入中的终结符。

但是，仅仅让空产生式返回空指针还不够。对于形如“零个或多个”的左递归列表规则，如果递归分支直接沿用左部已有节点，就可能在第一次出现真实元素时出现左部为空的情况。例如：

$$\text{ConstArrayDims} \rightarrow \varepsilon$$

$$\text{ConstArrayDims} \rightarrow \text{ConstArrayDims ConstArrayDim}$$

当第一次读到数组维度时，左侧的 `ConstArrayDims` 可能来自空产生式，因此其语义值为空。如果此时直接在空指针上追加子节点，就会导致数组维度信息无法进入语法树。

因此，实验中在这类递归规则中加入了判断：如果递归分支的左侧节点为空，则先创建对应的列表节点，再将当前真实元素挂接进去；如果左侧节点已经存在，则继续向该节点追加新的子节点。这样既避免了空产生式产生非终结符叶子，又保证了在实际存在内容时，列表结构仍然能够被正确保留。

通过这一处理，语法树在结构上更加符合实验要求：没有实际输入符号对应的空结构不会形成多余叶子，而数组维度、参数列表、语句列表等非空结构仍然能够以清晰的层次形式出现在树中。

3 实验结果

本实验最终实现了一个能够对 SysY 语言子集进行语法分析的分析器。对于语法正确的输入程序，分析器能够完成正常归约，输出分析过程中使用到的文法规则序列，并生成对应的语法树；对于包含部分语法错误的输入程序，分析器能够给出错误位置提示，并在部分语句或声明结构中进行局部恢复。

为了方便测试，在 `workspace` 目录下编写了自动化运行脚本 `run.sh`。该脚本能够自动完成词法分析器生成、语法分析器生成、编译、运行以及结果整理等步骤。例如，在 `workspace` 目录下运行：

```
./run.sh demo
```

可以对 `demo-code/test.sy` 进行测试；运行：

```
./run.sh example
```

可以对 `example/test.sy` 进行测试；运行：

```
./run.sh test1
```

则可以对 `test-cases/1.sy` 进行测试。类似地，`test2` 和 `test3` 分别对应 `test-cases` 目录下的其他测试文件。

每次运行脚本后，测试结果都会被统一保存到 `workspace/output` 目录下，并按照测试名称建立对应子目录。例如，运行 `./run.sh example` 后，输出结果会保存在：

```
workspace/output/example
```

其中主要包括输入文件副本 `input.sy`、标准输出文件 `stdout.txt`、错误输出文件 `stderr.txt`、归约规则序列文件 `rules.txt`、语法树描述文件 `tree.dot` 以及由 Graphviz 生成的语法树图片 `tree.png`。

从实验结果来看，分析器能够根据输入程序输出完整的归约过程，说明改写后的文法规则能够被 Yacc 正确用于自底向上的语法分析；生成的 `tree.dot` 和 `tree.png` 能够直观展示程序的语法层次结构，说明语法树节点构造与归约过程保持了一致；对于带有错误的测试样例，错误信息能够被输出到 `stderr.txt`，并且部分错误结构能够在语法树中以错误节点形式体现，说明错误恢复机制能够在一定范围内发挥作用。

总体而言，实验结果表明，本实验完成了词法分析器与语法分析器的对接、SysY 文法的 Yacc 化改写、归约序列输出、语法树构建、Graphviz 可视化以及基础错误恢复等功能，达到了语法分析实验的主要要求。